MIGRATING FROM MONOLITH TO MICROSERVICES

A Practical Guide for Backend Engineering Teams

Presented by Alex Chen, Principal Engineer

Platform Architecture Team | March 2026

Current State: The Monolith Problem

Deployment Bottleneck

- Single deployable artifact (~45 min build)
- Release train blocks all teams on slowest feature
- Rollbacks require full redeploy of entire app

Blast Radius

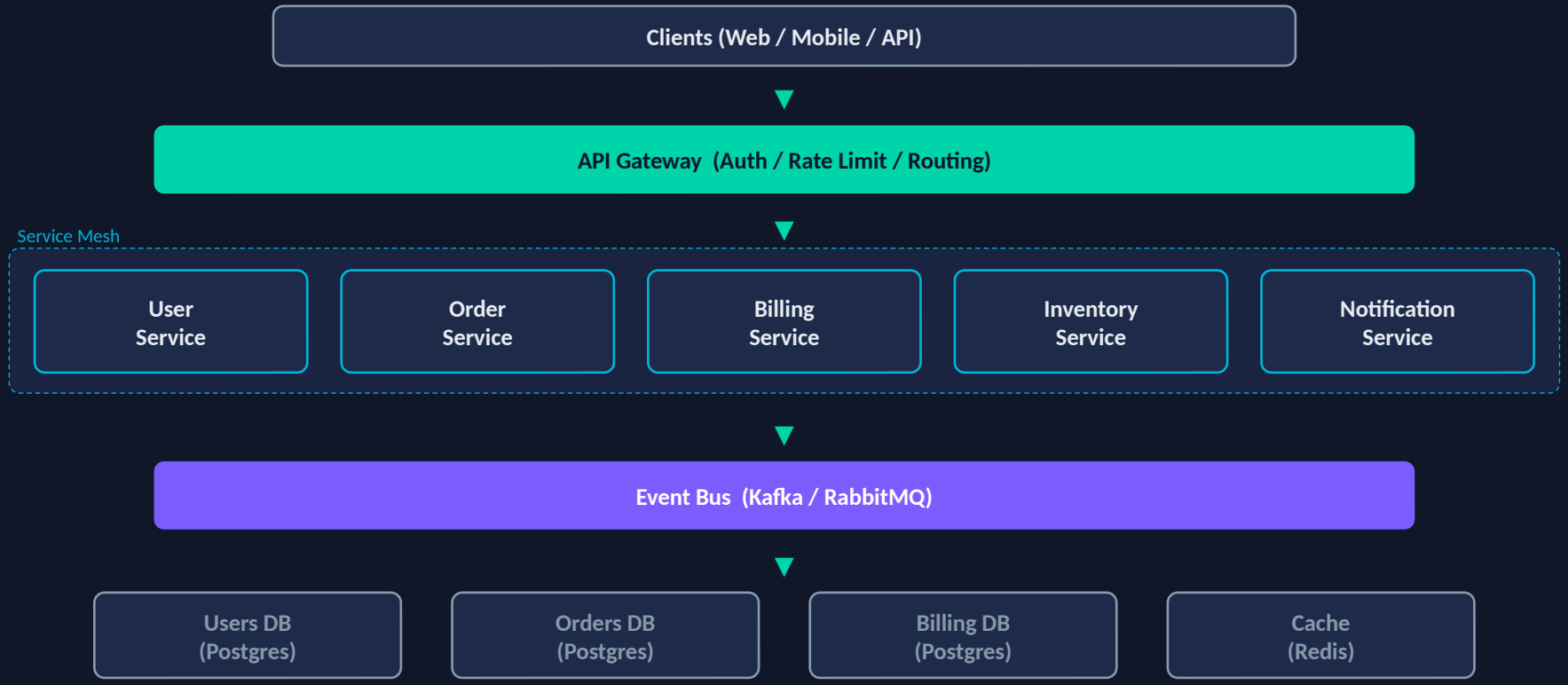
- OOM in billing module takes down entire platform
- No fault isolation between subsystems
- One bad deploy = full outage for all customers

Team Coupling

- 20 engineers contending on single codebase
- Merge conflicts average 3.2/day across teams
- Shared DB schema makes independent changes impossible

Current stats: Deploy frequency: 1x/week | MTTR: 4.2 hours | Lead time: 12 days

Target Architecture



Migration Strategy: Strangler Fig Pattern

Phase 1

Edge Proxy

Deploy API gateway in front of monolith. Route all traffic through it. Zero behavior change.

Phase 2

Extract Read Paths

Move read-only endpoints to new services. Dual-read validation ensures consistency.

Phase 3

Extract Write Paths

Migrate write operations with event sourcing. Use change data capture for sync during transition.

Phase 4

Decommission

Redirect remaining routes. Archive monolith codebase. Remove legacy DB dependencies.

Each phase includes rollback capability and feature flags for gradual cutover

Risk Matrix

Risk	Likelihood	Impact	Mitigation
Data inconsistency during dual-write phase	High	High	Event sourcing + CDC with reconciliation jobs
Service discovery failures	Medium	High	Consul mesh with health checks + circuit breakers
Cascading failures across services	Medium	High	Bulkhead pattern + timeout policies + fallback caching
Team skill gaps with distributed systems	High	Medium	Pairing program + internal training + embedded SRE
Performance regression from network calls	Medium	Medium	gRPC for internal comms + connection pooling + caching
Increased operational complexity	High	Medium	GitOps + unified observability platform from day one

Timeline: 6-Month Phased Rollout

Phase 1

Edge Proxy

- Deploy API gateway
- Configure routing rules
- Instrument baseline metrics
- Zero behavior change

Phase 2

Read Paths

- Extract user profile reads
- Extract catalog queries
- Dual-read validation
- Shadow traffic testing

Phase 3

Write Paths

- Migrate order writes
- CDC for data sync
- Event sourcing setup
- Feature flag cutover

Phase 4

Decommission

- Redirect final routes
- Archive monolith repo
- Remove legacy schemas
- Post-migration audit

Monitoring & Observability: Day One Setup

ALERTING STRATEGY

Metrics (Prometheus + Grafana)

- Request rate, error rate, duration (RED)
- Service-level SLIs/SLOs per endpoint
- Resource utilization per container

Logging (ELK Stack)

- Structured JSON logs with correlation IDs
- Centralized aggregation with log levels
- Automated alerting on error spikes

Tracing (Jaeger / OpenTelemetry)

- Distributed trace propagation
- Latency breakdown per service hop
- Dependency mapping auto-discovery

P1 - Critical

Service down, SLO breach > 5min
PagerDuty + auto-escalation

P2 - Warning

Error rate > 1%, latency p99 > 500ms
Slack alert + on-call review

P3 - Info

Deployment events, scaling triggers
Dashboard annotation only

Key Dashboards

- Service Health Overview (RED metrics)
- Infrastructure Resource Utilization
- Business KPI Real-time Feed
- Deployment & Rollback Tracker

Team Structure: Service-Oriented Squads

Platform Team (4 engineers)
API Gateway | Service Mesh | CI/CD | Observability

▼

Provides shared infrastructure to all squads

User Squad
4 eng

User Service
Auth Service

Commerce Squad
5 eng

Order Service
Billing Service

Inventory Squad
4 eng

Inventory Service
Catalog Service

Comms Squad
3 eng

Notification Svc
Email Service

Principles: You build it, you run it | Two-pizza teams | Service ownership = on-call ownership

Success Metrics

Deploy Frequency

BEFORE **1x / week**

AFTER →**Multiple / day**

Target at 6-month mark

Mean Time to Recovery

BEFORE **4.2 hours**

AFTER →**< 15 minutes**

Target at 6-month mark

p99 Latency

BEFORE **1200 ms**

AFTER →**< 200 ms**

Target at 6-month mark

DORA METRICS TARGETS

Metric	Current	Target	Notes
Change Lead Time	12 days	< 1 day	Time from commit to production
Change Failure Rate	23%	< 5%	Percentage of deploys causing incidents
Service Availability	99.5%	99.95%	Measured per-service, not monolith-wide

Questions?

Let's discuss architecture decisions,
migration concerns, and team readiness.

Alex Chen | alex.chen@company.dev | #platform-arch

Deck + reference materials: wiki.internal/microservices-migration